FICHE D’INVESTIGATION DE FONCTIONNALITÉ

Fonctionnalité: Recherche dans l’input principal	**Fonctionnalité #1**
Problématique: Accéder rapidement à une recette correspondant à un besoin de l’utilisateur dans les recettes déjà reçues	

Option 1: Utilisation de la programmation fonctionnelle Pour cette option, le filtre principal de recherche nous renvoie un nouveau tableau dont les éléments ont été parcourus grâce à la méthode map(). Les éléments sont filtrés et doivent répondre à une condition. Ces 2 actions sont possibles grâce à l’utilisation des méthodes filter() et (includes().	
Avantages: + La méthode map() est chaînable (possibilité d’utiliser les méthodes filter(), sort(), reduce() + Un code clair et concis + Rapidité d'exécution du code légèrement plus rapide + Un code plus “actuel” et facilement maintenable	**Inconvénients:** - L’utilisation de la programmation fonctionnelle est plus récente que celle des boucles natives
° **Nombre de caractères minimum à taper pour lancer la recherche:** 3 ° **La barre principale permet de rechercher des mots ou groupes de lettres dans:** le titre, les ingrédients ou la description	

Option 2: Utilisation des boucles natives

Pour cette seconde option, le filtre principal de recherche va parcourir le tableau grâce à la boucle native for. Ici, chacune des valeurs sera analysée et comparée à la condition prédéfinie. Si correspondance il y a alors, le résultat sera ajouté au tableau par le biais de la méthode générique push().

Avantages:	**Inconvénients:**
+ Elles sont connues par un plus grand nombre de développeurs car elles sont plus anciennes + Elles sont plus communément employées + Facilité d'écriture	- Elle n'est pas chaînable - Un code plus long, moins clair et obsolète - Une rapidité d'exécution légèrement inférieure

° **Nombre de caractères minimum à taper pour lancer la recherche:** 3
° **La barre principale permet de rechercher des mots ou groupes de lettres dans:** le titre, les ingrédients ou la description

Solution retenue:
Nous retiendrons l'utilisation de la programmation fonctionnelle car elle permet d'avoir un code plus "actuel" , maintenable, lisible et plus rapide d'exécution, notre but premier étant ce dernier point.

Annexes

Document 1: Extrait du code

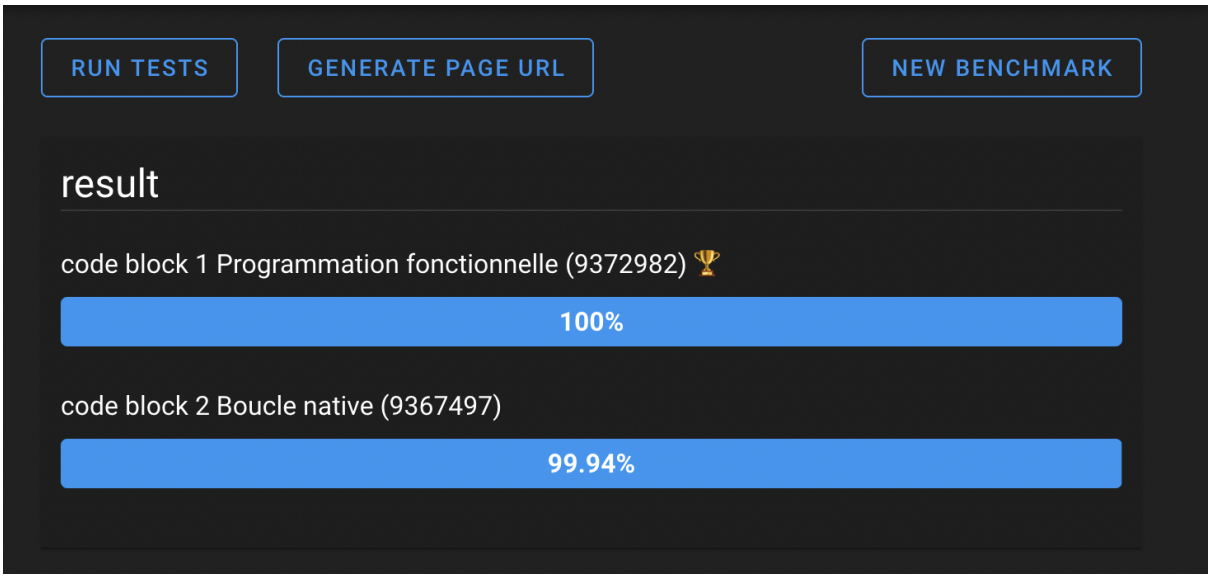
1-1 Boucle native

```javascript
// Filtre principal
function principalFilterNativeLoop(){
  // Récupérer la saisie de l'utilisateur
  let array = [];
  const wordSearched = document.getElementById('recherche_principale-input').value.toLowerCase();
  let names, descriptions, ingredients;

    for (let i = 0; i < recipes.length; i++) {
      names = recipes[i].name;
      descriptions = recipes[i].description;
          if(names.toLowerCase().includes(wordSearched)){
            array.push(recipes[i]);
          };
          if(descriptions.toLowerCase().includes(wordSearched)){
            array.push(recipes[i]);
          };

        for (let j = 0; j < recipes[i].ingredients.length; j++) {
          ingredients = recipes[i].ingredients[j].ingredient

          if (ingredients.toLowerCase().includes(wordSearched)){
            array.push(recipes[i]);
          }
        }
  }
  array = [... new Set(array)]
  return array;
};
```

1-2 Programmation fonctionnelle

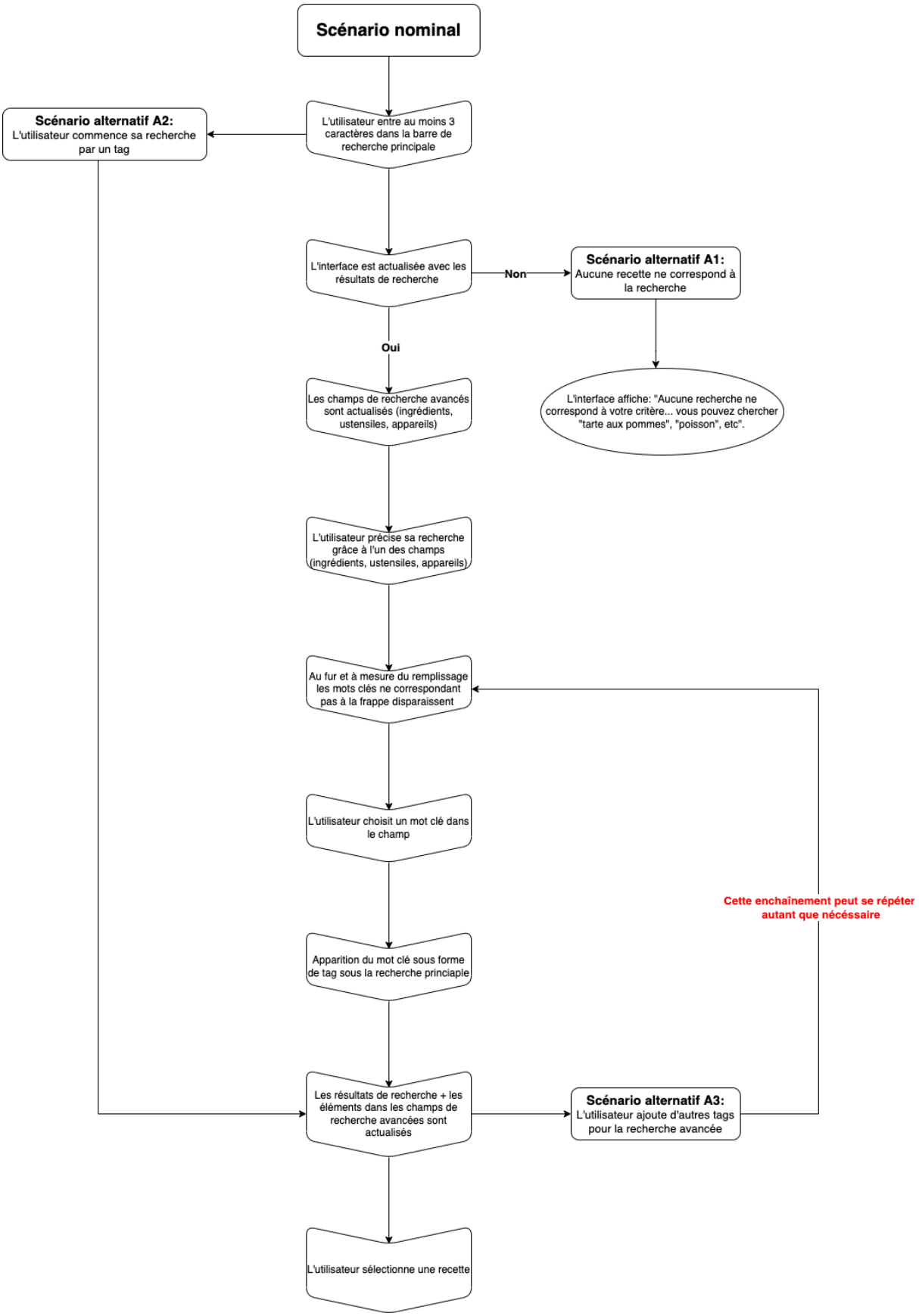
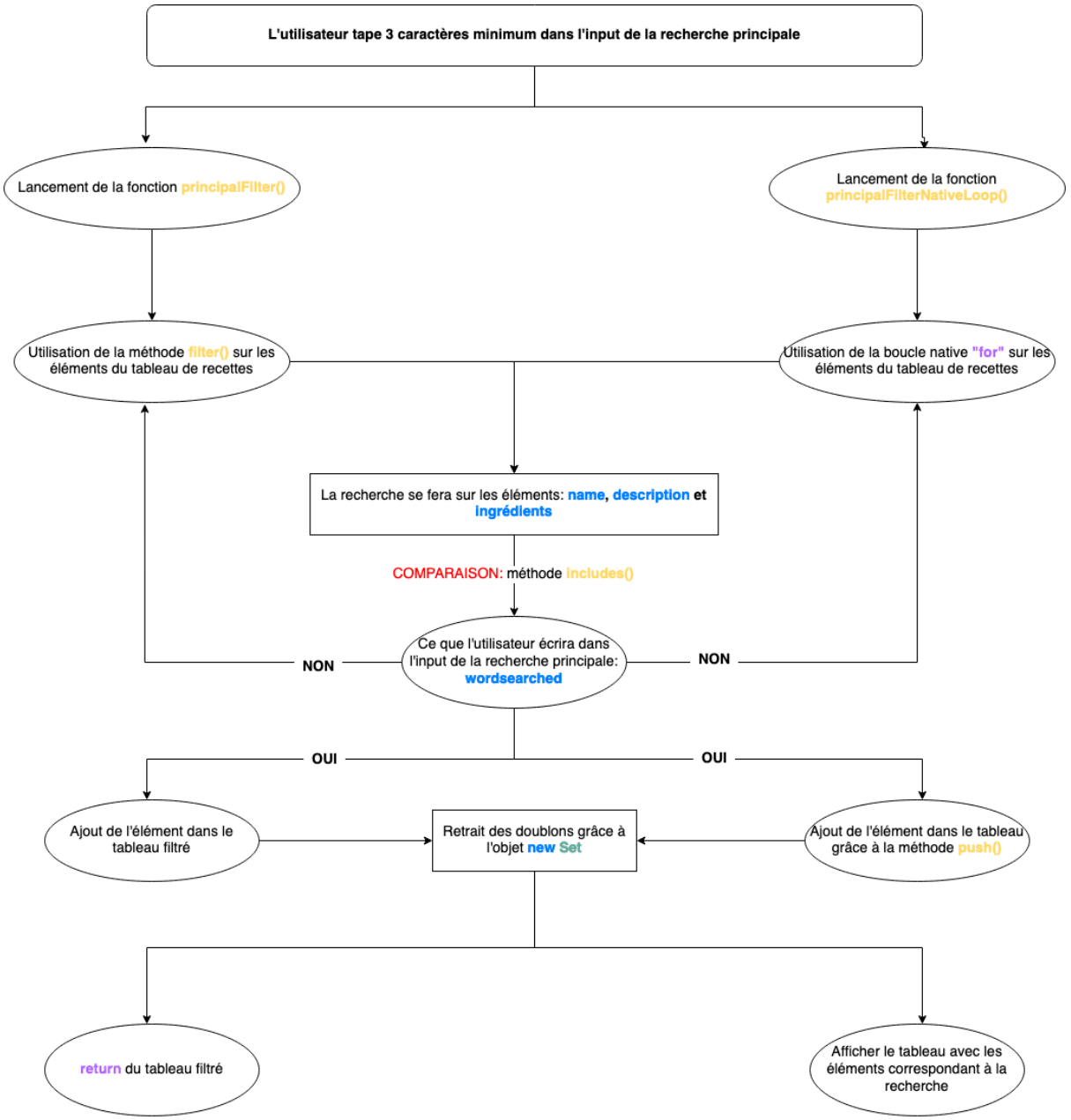
```javascript
// Filtre principal
function principalFilter(){
  // Récupérer la saisie de l'utilisateur
  const wordSearched = document.getElementById('recherche_principale-input').value.toLowerCase();
  let filteredRecipes = recipes;

    if(wordSearched.length > 2){
      filteredRecipes = recipes.filter(recette => {
        let titleDescritionIngredients = recette.name + recette.description + recette.ingredients.toString();
        if(titleDescritionIngredients.toLowerCase().includes(wordSearched)){
          return recette
        };
      });
  };
  return filteredRecipes;
};
```

Document 2: Résultats de l’analyse Jsben.ch



Document 3: Algorigrammes

3-1 Algorigramme des scénarios de la recherche principale



3-2 Algorigramme des filtres